GammaZero: Learned Value Aggregation

Design Document

David Shin

March 20, 2026

Contents

1 Motivation **2**

2 Architecture Overview **2**
2.1 Main Neural Network . . . 2
2.2 Latent Projection ($\mathbf{z}$) . . . 2
2.3 Aggregation Network . . . 3

3 Training Targets **4**
3.1 Target for $Q(p)$ . . . 4
3.2 Target for $U(p)$ . . . 4
3.3 Target for $W(p)$ . . . 5
3.4 Targets for AV and AU . . . 5
3.5 Prior-Consistency Regularization . . . 5

4 Training Procedure **6**
4.1 Sampling for $\mathcal{A}_\phi$ . . . 6
4.2 Bootstrapping and Regularization . . . 6

5 Inference and Search Integration **6**
5.1 Computational Cost . . . 6
5.2 Use in PUCT . . . 7
5.3 Future Direction: Replacing PUCT . . . 7

6 Spatial Latent Representations **8**

7 Stochastic Games **8**

8 Hyperparameters **9**

9 Open Questions **9**

1. Motivation

AlphaZero aggregates child value estimates into a parent estimate using a visit-count-weighted average:

$$Q(p) = \frac{V(p) + \sum_i N_i Q_i}{1 + \sum_i N_i}.$$

This is a reasonable heuristic but has well-known theoretical weaknesses. It conflates *exploratory* visits (induced by PUCT's exploration term and auxiliary mechanisms like policy temperature and Dirichlet noise) with *confirmatory* visits (made because a child is believed to be good). A child that was heavily explored but ultimately found to be poor will still exert downward pressure on $Q(p)$ proportional to its visit count. Additionally, a child suddenly found to be very good needs many more visits to offset highly-visited-but-inferior siblings than should be theoretically necessary.

GammaZero replaces this hand-crafted aggregation with a **learned aggregation network** $\mathcal{A}$ that takes children statistics as input and outputs a posterior parent estimate $Q(p)$. The hope is that $\mathcal{A}$ learns to discount exploratory visits and produce better-calibrated parent estimates, which in turn improves the quality of search.

2. Architecture Overview

2.1. Main Neural Network

The main NN f_θ maps a game state s to:

- **Policy head** P: prior move probabilities (same as AlphaZero).
- **Value head** V: prior scalar value estimate (same as AlphaZero).
- **Uncertainty head** U: prior scalar uncertainty estimate (based on KataGo; see Section 3.2).
- **Action-value head** AV: per-action value predictions (new; see Section 3.4).
- **Action-uncertainty head** AU: per-action uncertainty predictions (new; see Section 3.4).
- **Latent representation z**: a compressed projection of the trunk features, stored at the node upon expansion (see Section 2.2).

The main NN is evaluated *once per node expansion*, exactly as in AlphaZero.

2.2. Latent Projection (z)

The aggregation network conditions on positional context from the main NN. If the full trunk features were stored at each node, the memory cost would be prohibitive. To address this, a small **projection head** maps the trunk features down to a compact representation $\mathbf{z}$.

The simplest option is a **flat** projection to a vector of fixed dimension d_z (e.g., 32–64), capturing global positional properties such as tactical sharpness or game phase. An alternative is a **spatial** projection that retains board structure; see Section 6 for further discussion.

During training, there is only one network $\mathcal{N}$ comprising the combination of f_θ and $\mathcal{A}_\phi$. The portion pertaining to $\mathcal{A}_\phi$ is merely a head H_ϕ, emanating from the network trunk. At export

time, H_ϕ is detached from the rest of $\mathcal{N}$, resulting in separate networks f_θ and $\mathcal{A}_\phi$, with the $\mathbf{z}$ projection living in f_θ. This means the forward pass of f_θ produces the compact $\mathbf{z}$ directly, and only the small representation is stored at the node. $\mathcal{A}_\phi$ at inference time never sees the full trunk features.

2.3. Aggregation Network

The aggregation network $\mathcal{A}_\phi$ is a lightweight network evaluated at every *backup step*. Its outputs are stored at the node and read during selection (see Section 5.2). The outputs for a node p are:

- $Q(p)$: posterior value estimate for the parent node.
- $W(p)$: posterior uncertainty estimate for the parent node.

The inputs for a node p with children $\{c_1, \ldots, c_k\}$ are:

- Per-child statistics: $\{Q_i, W_i, N_i\}$ for each child c_i. For expanded children ($N_i > 0$), Q_i and W_i are outputs of $\mathcal{A}_\phi$ previously computed at child c_i. For unexpanded children ($N_i = 0$), $Q_i = AV_i$ and $W_i = AU_i$, the per-action predictions from f_θ (see Section 3.4).
- The prior outputs of f_θ at p: $\{P(p), V(p), U(p), AV(p), AU(p), \mathbf{z}(p)\}$.
- Baseline aggregates derived from the Law of Total Expectation (LoTE) and the Law of Total Variance (LoTV):

$$
\begin{aligned}
Q^*(p) &= \frac{V(p) + \sum_i N_i Q_i}{1 + \sum_i N_i} \\
W^*(p) &= \frac{U(p) + (V(p) - Q^*(p))^2 + \sum_i N_i (W_i + (Q_i - Q^*(p))^2)}{1 + \sum_i N_i}
\end{aligned}
$$

  The LoTV baseline $W^*(p)$ is motivated by interpreting W as a variance. This interpretation is loosely justified: the W target is defined as a squared difference between Q values (see Section 3.2), which matches the structure of variance computation. The interpretation need not be rigorous — $W^*(p)$ serves only as an input suggestion that $\mathcal{A}_\phi$ is free to use or ignore.

Because $\mathcal{A}_\phi$ is called at every backup step (typically $O(n \log n)$ times in a search tree with n nodes), it must be small. The latent $\mathbf{z}$ provides positional context, allowing $\mathcal{A}_\phi$ to modulate its aggregation strategy based on the character of the position (e.g., tactical sharpness, game phase).

Note that the per-child inputs Q_i and W_i for expanded children are themselves outputs of $\mathcal{A}_\phi$ from earlier backup steps lower in the tree. The aggregation network's inputs thus include its own prior outputs, giving the system a recursive structure. During training, gradients are stopped at this boundary: the child-level Q_i and W_i values are treated as fixed inputs.

3. Training Targets

3.1. Target for $Q(p)$

The training target for $Q(p)$ is the game outcome Z. This is the same supervision signal used for the value head V. The framing is:

- $V(p)$ is a **prior** value estimate, conditioned only on the board state.
- $Q(p)$ is a **posterior** value estimate, additionally conditioned on observed children statistics from search.

Since the network can always represent the function "output $Q^*(p)$" as a special case, it can in principle recover AlphaZero's aggregation. The hope is that dynamic children statistics carry additional signal about the true value.

3.2. Target for $U(p)$

The main NN includes a dedicated **uncertainty head** $U(p)$ that predicts **uncertainty**: how much the value estimate is expected to swing between the current position and the end of the game. For this, we adapt the target that KataGo uses for its Uncertainty-Weighted MCTS Playouts mechanism[1]. Specifically, at turn T:

$$U(p)_{\text{target}} = \big(V(p) - (1-\lambda)\sum_{t>T} Q_t \lambda^{t-1-T}\big)^2,$$

where Q_t represents the final value of Q of the tree root at the end of turn t, and where λ is a system hyperparameter. Note that the sum ranges over $t > T$, while KataGo's sum ranges over $t \geq T$. See Section 3.3 for the rationale.

$U(p)$ is predicted from board features alone, serving as the *prior* on uncertainty — exactly analogous to how V is the prior on value. $U(p)$ is passed as an input to the aggregation network, providing a baseline that the posterior $W(p)$ can refine.

The critical grounding property of this target is that $U(p) = 0$ for any position whose game-theoretic value is fully resolved — either because it directly precedes a terminal state, or because its subtree has been exhaustively searched (e.g., proven mate, proven draw). This gives the aggregation network a reliable anchor: $W_i = 0$ means child c_i's value Q_i is completely trustworthy, and $W_i = \epsilon$ means it is nearly so.

As noted in Section 2.3, we interpret $U(p)$ as a variance of $V(p)$ to compute a LoTV aggregation over children statistics. This is merely a chosen *interpretation* and is not rigorously justified. It need not be rigorously justified, as the LoTV-computed aggregation serves merely as a *suggestion* to $\mathcal{A}_\phi$ on how to compute the actual uncertainty-aggregation, which $\mathcal{A}_\phi$ is free to use or ignore. Whether the target definition or the LoTV aggregation are useful are empirical questions; what ultimately matters is whether the choices help $\mathcal{A}_\phi$ produce better $Q(p)$ estimates.

[1] See: `https://github.com/lightvector/KataGo/blob/master/docs/KataGoMethods.md#uncertainty-weighted-mcts-playouts`

3.3. Target for $W(p)$

$W(p)$ is the posterior version of $U(p)$, output by the aggregation network. It is trained to predict an analogous target to $U(p)$, but with the benefit of access to children statistics. Denoting by $Q^{(n)}(p)$ the output of $\mathcal{A}_\phi$ after n search iterations:

$$W(p)_{\text{target}} = \big(Q^{(n)}(p) - (1-\lambda)\sum_{t>T} Q_t \lambda^{t-1-T}\big)^2,$$

As for why our sum ranges over $t > T$ rather than $t \geq T$: the expected difference between the current $Q^{(n)}(p)$ and Q_T will depend on n, the search iteration for which the sample was obtained. This would introduce an inappropriate coupling between n and $W(p)_{\text{target}}$.

The LoTV-derived baseline $W^*(p)$ serves as a regularization anchor during early training. See Section 4.2.

3.4. Targets for AV and AU

The main NN includes per-action heads $AV(s,a)$ and $AU(s,a)$ that predict the value and uncertainty at the child state reached by action a. Their training targets are the main NN's own evaluations at those child states:

$$AV(s,a)_{\text{target}} = V(s'),$$
$$AU(s,a)_{\text{target}} = U(s'),$$

where s' is the state reached by playing action a from s.

To produce these targets, children of sampled root nodes are expanded during training. This can be done exhaustively (expanding all children) or by sampling a random subset and computing loss only on that subset.

AV and AU serve as the default Q_i and W_i inputs for unexpanded children when feeding into $\mathcal{A}_\phi$. This allows the aggregation network to reason about all children — not just those that have been visited — using the main NN's prior predictions as stand-ins.

Note that AlphaZeroArcade[2] already uses an AV head, whose output is used for the Q term in the PUCT formula for unvisited children.

3.5. Prior-Consistency Regularization

When all children are unexpanded ($N_i = 0$ for all i), the aggregation network has no search evidence to incorporate. The only child-level information available is the AV/AU priors. In this case, $\mathcal{A}_\phi$ has no information beyond what f_θ already captured, so the posterior should equal the prior:

$$Q(p) = V(p), \qquad W(p) = U(p) \qquad \text{when all } N_i = 0.$$

This is enforced as a regularization term during training: a fraction of training samples are presented with all $N_i = 0$, and a loss term encourages $\mathcal{A}_\phi$ to reproduce $V(p)$ and $U(p)$ in this regime.

[2] `https://github.com/shindavid/AlphaZeroArcade`

4. Training Procedure

4.1. Sampling for $\mathcal{A}_\phi$

Following KataGo's Playout Cap Randomization, some fraction of root nodes are sampled for NN training. Of these, a configurable fraction α is allocated to training the main NN f_θ, and the remainder $1-\alpha$ to training the aggregation NN $\mathcal{A}_\phi$. The split ratio α is a system hyperparameter; since f_θ is a larger network learning a harder function, $\alpha > 0.5$ is likely appropriate.

For positions allocated to $\mathcal{A}_\phi$ training, if N is the search budget in iterations, we uniformly sample some n from $\{1, \ldots, N\}$ and snapshot the tree statistics after n iterations have been performed. This means $\mathcal{A}_\phi$ learns to produce a useful posterior estimate given any amount of search evidence, from very little (n small) to nearly the full budget ($n \approx N$).

4.2. Bootstrapping and Regularization

Early in training, $\mathcal{A}_\phi$ is uninformed, which risks destabilizing the feedback loop between search quality and training targets. To mitigate this, the network is regularized during early training to match the hand-crafted baselines:

$$
\begin{aligned}
Q(p)_{\text{baseline}} &= Q^*(p), \\
W(p)_{\text{baseline}} &= W^*(p).
\end{aligned}
$$

This regularization is annealed away as training progresses. Because $Q^*(p)$ and $W^*(p)$ are inputs into $\mathcal{A}_\phi$, they are representable and thus constitute reachable regularization targets. The network will learn initially to simply directly output these inputs, providing some initial stability. As the annealing tapers, the network should gradually mature to produce more nuanced outputs.

The prior-consistency regularization (Section 3.5) provides an additional stability mechanism: even if the LoTE/LoTV regularization is annealed away, $\mathcal{A}_\phi$ retains sensible behavior in the zero-search regime.

5. Inference and Search Integration

5.1. Computational Cost

The main NN is evaluated once per node expansion, unchanged from AlphaZero. The aggregation network $\mathcal{A}_\phi$ is evaluated at every backup step. Its $Q(p)$ and $W(p)$ outputs are stored at the node and read during selection, requiring no additional evaluation at selection time. Inference cost is kept low by:

- Keeping $\mathcal{A}_\phi$ small (few linear layers).
- Reusing the stored latent $\mathbf{z}(p)$ rather than re-running the main NN.

In self-play mode, evaluations of $\mathcal{A}_\phi$ across simultaneous games are batched, amortizing cost. In single-game mode, we may be able to adopt a similar mechanism to standard MCTS parallelization, which involves multiple search threads and virtual visits. Some details remain to be worked out.

It may be possible to incorporate NNUE[3] mechanics. In a game like chess, the policy space is large, while only a sparse subset corresponds to legal moves in any given position. Furthermore, typically only one child will have changes to its statistics during aggregation (more than one is possible due to multithreaded search or due to move transpositions). These properties are the ingredients that make NNUE feasible in chess and Othello.

5.2. Use in PUCT

$\mathcal{A}_\phi$'s stored Q_i output for each child c_i is used directly in the PUCT selection formula:

$$\text{PUCT}_i = Q_i + c \cdot P_i \cdot \frac{\sqrt{1 + N(p)}}{1 + N_i}.$$

This is a departure from standard AlphaZero, where Q_i during selection is simply the running average $\text{total_value}_i / N_i$. Here, Q_i is the aggregation network's learned posterior estimate, computed during backup and cached at the node.

5.3. Future Direction: Replacing PUCT

Later, we may add a head to $\mathcal{A}_\phi$ that outputs a score that replaces PUCT entirely. The challenge is devising a training target that captures the purpose of PUCT.

One idea for this is to define a notion of "exploration worthiness". PUCT aims to identify actions worth exploring. What makes an action worth exploring? One candidate definition is to say that an action is worth exploring if doing so has strong potential for changing our current Q estimate. This suggests using a training target that is based on the change in $Q(p)$ induced by visiting a given child. A similar alternative is to say that an action is worth exploring if *not* doing so for the remaining search budget is likely to produce a different Q estimate than an unconstrained usage of the remaining search budget. Still another possibility is to locate exploration worthiness in a likely reduction of $W(p)$, rather than an expected delta in $Q(p)$. It may be possible to produce targets corresponding to these notions.

All these options have the property that we can use an exact score of 0 for fully resolved positions (e.g., proven mate, proven draw), while a NN estimated position can be made to return a strictly positive score. This has the nice property that the selection criterion would naturally avoid re-exploring fully resolved positions, a property that standard PUCT lacks.

If something can be found to work here, it may be possible to remove one or more heads among P, AU, and AV, as they exist primarily to facilitate the PUCT criterion.

[3] See `https://www.chessprogramming.org/NNUE`

6. Spatial Latent Representations

Section 2.2 introduced the latent projection $\mathbf{z}$ as a compressed representation of the trunk features. The simplest design is a flat vector of dimension d_z, which captures global positional properties but discards spatial structure. An alternative is to retain the board geometry by projecting to a **spatial tensor** of shape (H, W, r), where $H \times W$ matches the board dimensions and r is a small channel depth (e.g., 4–8).

A spatial $\mathbf{z}$ enables $\mathcal{A}_\phi$ to represent **correlation beliefs between actions**. For example, in Go on a 19×19 board, a spatial projection of shape $(19, 19, r)$ could learn to map each candidate move to a vector on an r-dimensional hypersphere, with the cosine similarity between two move-vectors encoding the aggregation network's belief about how correlated their values are. This would allow $\mathcal{A}_\phi$ to reason about questions like: "move A and move B both attack the same group, so if A's value dropped after search, B's value likely dropped too." A flat $\mathbf{z}$ cannot represent such per-action structure.

The memory cost of a spatial $\mathbf{z}$ is $(H \times W \times r)$ floats per node. For Go with $r = 8$, this is $19 \times 19 \times 8 = 2{,}888$ floats — reasonable for practical use.

This design is more natural for games with spatially structured action spaces (Go, Hex, Othello) than for games like Chess, where the action space is a set of (source, destination) pairs rather than single board intersections.

Whether spatial $\mathbf{z}$ actually helps is an empirical question. Even if the representation has the capacity to encode correlations, the aggregation network must learn to use them, and the training signal (game outcomes) may be too diffuse to drive fine-grained correlation learning. The flat design is the safe default; spatial $\mathbf{z}$ is an avenue for games where it is natural.

7. Stochastic Games

For games with explicit stochastic events whose probability distributions are known (e.g., dice rolls), the uncertainty target conflates epistemic uncertainty with aleatory uncertainty from the game mechanics. To compensate, a **luck correction** is applied using the main NN's value predictions at chance nodes.

In games with stochastic events, the search tree contains **chance nodes** (representing stochastic outcomes) alongside **player nodes** (representing player decisions). The main NN evaluates both node types, producing V predictions at each. At a chance node with outcome ω, the luck delta is:

$$\delta_\omega = V(\text{child}) - V(\text{parent}),$$

i.e., the shift in value prediction between the pre-event state and the post-event state. If per-action value heads are available, this simplifies to $\delta_\omega = V(\text{child}) - AV(\text{parent}, a_\omega)$, where a_ω is the action corresponding to outcome ω.

The luck-adjusted Q along a trajectory is then:

$$Q_{\text{adj}} = Q_{\text{raw}} - \sum_{\text{events}} \delta_\omega.$$

This is a martingale correction: it removes the random-walk component from the value trajectory, leaving only the skill/position component. The $U(p)$ and $W(p)$ training targets are then computed against Q_{adj} rather than Q_{raw}.

No separate luck estimator network is required — the correction falls out of the main NN's existing value predictions at chance nodes.

For deterministic games (Chess, Go, Hex, Connect4), there are no chance nodes and this section does not apply.

8. Hyperparameters

Table 1 summarizes the tunable hyperparameters of the system.

Symbol	**Description**	**Default / Suggested**
α	Fraction of sampled roots allocated to f_θ training	> 0.5 (TBD)
λ	Exponential decay for uncertainty targets	5/6 (from KataGo)
d_z	Projection dimension (flat $\mathbf{z}$)	32–64
r	Channel depth (spatial $\mathbf{z}$)	4–8
	Annealing schedule for LoTE/LoTV regularization	TBD
	Fraction of $N_i = 0$ samples for prior-consistency	TBD

Table 1: System hyperparameters.

9. Open Questions

1. **Aggregation network architecture and inference mechanics.** How many layers? NNUE? How does query batching work in single-game mode?
2. **Regularization details.** There are multiple components of annealing and regularization (LoTE/LoTV baseline, prior-consistency, and potentially independent annealing rates for Q vs. W). The interactions between these components need to be explored empirically.
3. AV/AU **training efficiency.** Expanding all children at sampled roots may be expensive for games with large branching factors. The subset-sampling alternative introduces variance; the right balance is to be determined.
4. **Empirical validation** on Connect4, then Othello, Hex, Chess.